

UNIT 1: DETAILED MINI PROJECT REPORT

AI-Powered Smart Campus
Helpdesk & Incident
Management System

DevOps and Cloud Foundation (IBM)

STUDENT DETAILS	INFORMATION
Student Name	Nikhil Shidore
Department	Artificial Intelligence & Data Science (AIDS)
PRN	2126UDSM2575
Subject	DevOps and Cloud Foundation (IBM)
Scope	Unit 1 – Product Engineering, IBM Design Thinking, Architecture & Agile Delivery Plan

PROJECT FOCUS

- Cloud-native campus helpdesk
- AI-assisted support using IBM Watson
- IBM Cloudant JSON database
- Automated routing, SLA tracking and CI/CD

1. Product Engineering and SDLC Framework

Software product engineering is the whole process of understanding the user problem, designing a useful product, testing it, deploying it, and continuously improving it on the cloud. Coding is one execution phase inside a larger engineering lifecycle that also includes research, architecture, user experience, quality assurance, automation, security, monitoring and feedback.

1.1 Programming vs. Software Product Engineering

Aspect	Programming / Coding	Product Engineering
Main Goal	Write code for a function or module.	Solve a real user problem through a reliable product.
Work Scope	Syntax, algorithms and debugging.	Research, design, architecture, microservices, cloud, security and CI/CD.
Success Criteria	Code runs without error.	System is useful, secure, scalable, maintainable and available.
Mindset	Technology-centric: “How do I write this?”	Product-centric: “What problem are we solving?”
Lifecycle	Usually focused on implementation.	Covers requirements, design, build, test, deploy and feedback.

1.2 Root Causes of Software Project Failures

- Unclear or misaligned requirements: features may be built that students, faculty or staff do not actually need.
- Technology-centric bias: a tool may be selected because it is popular rather than because it suits the campus problem.
- Communication silos: business users, developers, QA and DevOps teams may work without enough coordination.
- Late feedback: if users test the system only near the end, changes become slower and more expensive.
- Insufficient monitoring: without operational feedback, recurring incidents and performance problems may remain hidden.

1.3 Engineering Approach for Smart Campus Helpdesk

- Define the campus user problem before implementation.
- Design cloud architecture and data flow before writing major modules.
- Automate testing and deployment early through CI/CD.
- Use role-based access, secure authentication and controlled data storage.
- Monitor ticket flow, response time and overdue incidents.
- Collect feedback and continuously improve the product.

2. Software Development Lifecycle (SDLC)

The Smart Campus Helpdesk follows a structured lifecycle so that business understanding, design, implementation, testing and operations remain connected. The reference framework uses six major stages.

Stage	Project Activity	Output
1. Requirements & Domain Analysis	Identify campus issues, user needs and stakeholder expectations.	Requirement specification
2. IBM Design Thinking & Wireframes	Observe users, study pain points and create prototypes.	Personas, empathy map, wireframes
3. Cloud Architecture & Database Design	Select cloud services and design the ticket data model.	Cloud architecture + database design
4. Implementation & Microservices	Develop frontend, APIs, ticket logic and AI integration.	Working modules
5. Testing & CI/CD	Run automated tests and build the GitHub Actions pipeline.	Validated build
6. Deployment, Monitoring & Feedback	Deploy, observe system behavior and collect feedback.	Cloud MVP + improvement loop

2.1 Detailed SDLC Stage Explanation

Stage 1 – Requirements – The team identifies problems such as Wi-Fi login failures, lab computer issues and projector faults. Functional and non-functional requirements are documented before implementation.

Stage 2 – Design Thinking – Users are observed and their frustrations are converted into personas, problem statements and low-fidelity wireframes. This reduces the risk of building an unusable interface.

Stage 3 – Architecture – The team defines frontend, backend, AI service, Cloudant database, Object Storage, deployment platform and CI/CD connections.

Stage 4 – Implementation – Authentication, ticket CRUD operations, Watson integration, routing and dashboard modules are implemented as working application components.

Stage 5 – Testing & CI/CD – Automated tests validate important behaviors. GitHub Actions can build and validate the application consistently after code changes.

Stage 6 – Deployment & Feedback – The application is deployed to the cloud, monitored and evaluated using user feedback. Improvements are then fed back into the next iteration.

KEY IDEA

The lifecycle is iterative: deployment does not end engineering. Monitoring and feedback provide inputs for the next improvement cycle.

3. Domain & Business Understanding

Before writing code or provisioning cloud infrastructure, engineers must understand how campus support currently works. A helpdesk connects students and faculty who report issues with staff who classify, assign, resolve and track those issues.

3.1 Core Campus Helpdesk Operations

- **Incident Reporting:** Students and faculty submit issues such as Wi-Fi, lab systems, projector or access problems.
- **Ticket Classification:** The issue is categorized by type, department and priority.
- **Assignment & Routing:** Tickets are assigned to the appropriate IT or support staff member.
- **SLA Tracking:** Priority and due-time information are tracked so overdue incidents can be identified.
- **Resolution & Closure:** Staff update status, add resolution information and close the ticket.
- **Notification & Communication:** Users receive status information and can view ticket progress.
- **Analytics & Administration:** Admins monitor active, resolved and overdue tickets and review operational trends.

3.2 Manual Challenges vs. Smart Platform

Manual Operational Bottleneck	Smart Campus Solution	Business Outcome
Problem reported through paper/email	Online web portal + IBM Watson virtual assistant	Centralized reporting and faster access
Staff manually sort requests	Auto-routing by category/priority	Reduced assignment delay
Users repeatedly ask for status	Live ticket status and notifications	Better transparency
Data stored in Excel/paper	IBM Cloudant JSON documents	Centralized and structured records
Manual overdue tracking	SLA/priority tracking and dashboard	Easier identification of overdue tickets
Screenshots shared separately	Cloud Object Storage attachment link	Evidence remains linked to the ticket
Manual counting of incidents	Live admin dashboard	Faster operational reporting

3.3 Business Justification

- Reduce response time for common campus issues.
- Prevent missed or lost tickets through centralized tracking.
- Provide 24/7 automated help for configured FAQs.
- Give staff a clear view of priority and overdue incidents.
- Create a cloud-native foundation that can be expanded later.

4. Introduction to IBM Design Thinking

Design Thinking is an iterative, human-centered problem-solving framework. It prioritizes understanding users and their pain points before deciding what technology to build. For the Smart Campus Helpdesk, the approach helps the team design around real student, faculty and IT-support workflows.

4.1 Core Principles

- **Human-Centered Innovation:** Design the service around real user behavior instead of forcing users to adapt to rigid technology.
- **Creative Problem-Solving & Ideation:** Explore multiple possible solutions before selecting an implementation.
- **Iterative Prototyping:** Test simple wireframes or mockups early so problems can be discovered before expensive development.
- **Continuous Learning:** Use observations and feedback to refine requirements and user experience.

4.2 Five Stages of Design Thinking

Stage	Meaning	Smart Campus Application
Empathize	Study users and pain points.	Observe students, faculty and IT staff reporting and resolving incidents.
Define	Formulate the core problem.	Define delays, lack of status visibility and manual routing as key problems.
Ideate	Brainstorm possible solutions.	AI FAQ assistant, ticket portal, auto-routing, dashboard and notifications.
Prototype	Create a simple representation.	Login, ticket form, Watson chat, dashboard and status wireframes.
Test	Validate with users.	Check navigation, clarity and whether users can report and track incidents easily.

4.3 Detailed Stage Application

Stage 1: Empathize – Key user categories are students, faculty and IT/support staff. The team studies what users say, think, do and feel when an incident occurs.

Stage 2: Define – A suitable core problem statement is: “Students and teachers face delays because campus incidents are reported and routed manually, while IT staff lack centralized real-time tracking of priority and overdue tickets.”

Stage 3: Ideate – Possible features include IBM Watson FAQ support, online ticket creation, automatic category-based routing, SLA tracking, dashboard analytics and notifications.

Stage 4: Prototype – Low-fidelity screens can include Login, Create Ticket, My Tickets, Watson Assistant, Staff Dashboard and Admin Reports.

Stage 5: Test – Students, faculty and staff can review the prototype to identify unclear fields, navigation problems and missing ticket information before frontend engineering.

5. IBM Design Thinking: Enterprise Expansion

IBM Design Thinking extends human-centered design into enterprise delivery by connecting user outcomes, continuous improvement and cross-functional teamwork.

5.1 Three Core Pillars

Pillar	Explanation	Project Application
Focus on User Outcomes	Shift from technical output to measurable user outcomes.	Instead of only “build ticket search,” aim to make ticket creation and status tracking simple and understandable.
Restless Reinvention	Continuously analyze usage and improve the product.	Use ticket trends and feedback to refine routing, FAQ content and dashboard design.
Diverse Empowered Teams	Bring multiple disciplines into delivery.	Product/domain stakeholders, UX, developers, QA, database and DevOps collaborate.

5.2 IBM Loop Engine - Observe → Reflect → Make

The IBM Loop Engine represents a continuous cycle rather than a one-time design exercise.

- **OBSERVE** – Study user interactions, ticket patterns, logs and operational friction.
- **REFLECT** – Analyze findings, review metrics and align priorities with user needs.
- **MAKE** – Design UI improvements, implement code, automate testing/deployment and release changes.
- **Repeat** – Return to observation after the updated system is used.

SMART CAMPUS EXAMPLE

Observe repeated Wi-Fi tickets → Reflect on common questions → Make a better Watson FAQ and routing rule → Observe whether repeated tickets decrease.

6. Stakeholder User Personas & Access Control

Personas represent typical users so the team can connect system features to real objectives and pain points.

Persona	Role	Objective / Need	Pain Point	Platform Feature
Aniket	Student	Report fast and check status.	No clear status update.	AI chat + live ticket progress
Prof. Sharma	Faculty	Get urgent projector issue fixed.	Delay during live class.	High-priority flag + notification
Rajesh	IT Staff	Resolve and assign tickets.	Duplicate/manual requests.	Dashboard + automated routing
Admin	Administrator	Monitor campus support.	Manual reporting.	Analytics dashboard + reports

6.1 Stakeholder Matrix

Stakeholder Group	Key Roles	Responsibilities & Expectations
Business Stakeholders	College management, students, faculty, admin	Define business vision, validate usability and confirm campus requirements.
Engineering Stakeholders	Developers, UI/UX designers, QA	Architect and build application modules, interfaces and automated tests.
Operations Stakeholders	DevOps, database and security teams	Provision cloud infrastructure, protect data and automate CI/CD.

6.2 Access Control Matrix

Feature	Student	Faculty	IT Staff	Admin
Talk to Watson AI	Yes	Yes	Yes	Yes
Create Ticket	Yes	Yes	Yes	Yes
View Own Tickets	Yes	Yes	Yes	Yes
Assign & Resolve Ticket	No	No	Yes	Yes
System Settings & Reports	No	No	No	Yes

7. Problem Statement, Solution & Product Vision

CORE PROBLEM STATEMENT

Students and teachers face long delays due to manual incident reporting and routing. IT staff lack centralized, automatic tracking of ticket priority, status and overdue work.

7.1 Selected Solution

Feature	Release	Purpose
Role-Based Login	MVP Release 1	Secure login for Student, Faculty and Admin/Staff roles.
IBM Watson Virtual Assistant	MVP Release 1	Instant 24/7 answers for configured common questions.
IBM Cloudant JSON Database	MVP Release 1	Flexible NoSQL storage for ticket history.
Automated SLA Routing	MVP Release 1	Assign tickets using priority/category/department rules.
Email & SMS Notifications	Release 2	Send status-change and alert messages.
Predictive Hardware Analytics	Future	Use analytics to identify possible equipment-failure patterns.

7.2 Product Vision Statement

“To deliver a modern, automated and cloud-native campus helpdesk that reduces manual administrative effort, streamlines incident reporting and routing, and provides an intuitive support experience for students, faculty and staff.”

7.3 Minimum Viable Product (MVP) Scope

- User Login and Role-Based Access Control.
- Ticket creation, viewing and basic status management.
- IBM Watson FAQ support.
- Cloudant-based ticket storage.
- Basic automated routing and priority handling.
- Admin dashboard for active, resolved and overdue tickets.

MVP PRINCIPLE

Non-essential capabilities can be deferred so that the first release remains focused, testable and useful.

8. Requirements Engineering Specification

Requirements describe what the system must do and the quality conditions under which it must operate. Separating functional and non-functional requirements makes development and acceptance testing clearer.

8.1 Functional Requirements (FRS)

- FR01 Authentication: Secure sign-in for Student, Faculty and Admin roles.
- FR02 AI Assistant: IBM Watson answers configured FAQs such as Wi-Fi setup and password reset.
- FR03 Ticket Management: Users create, view and close tickets; staff assign priority and handle tickets.
- FR04 File Attachment: Screenshots can be stored in IBM Cloud Object Storage and linked to tickets.
- FR05 Admin Dashboard: Show real-time total, active, resolved and overdue ticket information.
- FR06 Routing: Assign tickets using category, priority or department rules.
- FR07 Notifications: Future release can notify users about ticket status changes.

8.2 Non-Functional Requirements (NFRS)

- Security: HTTPS, JWT/session management and hashed passwords.
- Performance: Watson response target below 400 ms and database queries below 500 ms as specified in the project report.
- Availability: Target 99.9% cloud uptime on IBM Cloud Code Engine.
- Scalability: Cloud-native services should support increasing ticket volume without redesigning the whole application.
- Maintainability: Modular repository structure, automated tests and clear separation of configuration, modules and tests.
- Usability: Common tasks such as reporting and checking a ticket should be easy to understand.

8.3 Requirements Traceability Idea

Requirement	Validation Method	Expected Evidence
FR01 Login	Login test + role test	Correct users can sign in; unauthorized access is blocked.
FR02 Watson	FAQ scenario test	Configured question receives expected answer.
FR03 Ticket	Create/update test	Ticket is stored with required fields and status.
FR04 Attachment	Upload test	File stored and URL linked to ticket.
FR05 Dashboard	Dashboard validation	Counts and status categories are displayed.

9. System Architecture & Database Design

The system is organized into user-facing frontend components, backend services, AI integration and cloud data/storage layers. This separation helps keep responsibilities clear and supports future scaling.

ARCHITECTURE FLOW

USERS → FRONTEND → BACKEND/API SERVICES → AI + DATABASE/STORAGE → CLOUD DEPLOYMENT
→ MONITORING & FEEDBACK

9.1 Architecture Components

Component	Purpose
Frontend	Login, ticket creation, ticket status, Watson chat and dashboards.
Backend/API	Authentication, ticket CRUD, routing, reports and integration logic.
IBM Watson Assistant	Conversational support and FAQ handling.
IBM Cloudant	JSON document storage for users/tickets and related data.
IBM Cloud Object Storage	Stores screenshots and other ticket attachments.
IBM Cloud Code Engine	Containerized cloud deployment target.
GitHub Actions	Automated CI/CD build and validation workflow.

9.2 ERD & Data Design

Entity / Field	Meaning
USERS.user_id	Unique user identifier.
USERS.name / email	Basic user information.
USERS.role	Student, Faculty, IT Staff or Admin access role.
TICKETS.ticket_id	Unique incident/ticket identifier.
TICKETS.category	Type such as Network/Wi-Fi, Hardware or Classroom.
TICKETS.status	Current state such as OPEN, IN_PROGRESS or RESOLVED.
TICKETS.priority	Priority used for routing and SLA handling.
TICKETS.assigned_to	Staff member responsible for the ticket.
ATTACHMENTS.attachment_id	Unique attachment identifier.
ATTACHMENTS.ticket_id	Ticket reference for the attachment.
ATTACHMENTS.file_url	Object Storage location.

9.3 Sample Cloudant JSON Document

```
{
  "_id": "ticket::10482",
  "ticket_id": "INC-2026-0921",
  "created_by": {"user_id": "USR-8821", "name": "Aniket Patel", "role": "Student"},
  "category": "Network/Wi-Fi",
  "subject": "Unable to connect in CS Lab 3",
```

```
"status": "IN_PROGRESS",  
"priority": "HIGH",  
"assigned_to": "TECH-STAFF-04",  
"attachments": ["../helpdesk/screenshots/err_01.png"],  
"timestamps": {"created_at": "2026-09-21T09:30:00Z"}  
}
```

10. Agile Sprint Plan & Enterprise Engineering Roadmap

Agile delivery divides the project into small increments. Each sprint produces a practical deliverable while keeping the architecture and user feedback loop active.

Sprint	Work Scope	Deliverable
Sprint 1	Requirements, Cloudant setup, authentication	Working login + database connection
Sprint 2	Watson Assistant + ticket APIs	AI chatbot widget + ticket endpoints
Sprint 3	Admin dashboard, SLA routing, Object Storage	Ticket portal + file upload
Sprint 4	Automated testing, GitHub Actions CI/CD	Deployed application on IBM Cloud Code Engine

10.1 Enterprise Engineering Roadmap

- Phase 1: Domain Analysis & Problem Understanding
- Phase 2: Design Thinking & Prototyping
- Phase 3: Agile Sprint Planning & Backlog Management
- Phase 4: Core Application Development (Backend & Frontend)
- Phase 5: DevOps Automation (CI/CD Pipelines & Containerization)
- Phase 6: Infrastructure Automation and configuration management
- Phase 7: Production Deployment & Monitoring

10.2 Definition of Done - Example

- Feature implemented according to its requirement.
- Relevant automated test added and passed.
- Role/access behavior checked.
- Code committed through the repository workflow.
- Build/CI pipeline passes.
- Deployment and basic runtime behavior verified.

11. Repository Structure & Git Setup

A standardized repository structure separates application code, configuration, tests, documentation and deployment resources. This supports maintainability and collaboration.

```
smart-campus-helpdesk/
├── app/
│   ├── config/          # IBM Cloud API configuration
│   ├── modules/auth/    # Login & role control
│   ├── modules/tickets/ # Ticket CRUD functions
│   ├── modules/watson/  # Assistant integration
│   └── tests/           # Automated tests
├── docs/                # Diagrams & report
├── .github/workflows/   # GitHub Actions CI/CD
├── docker/Dockerfile    # Container configuration
└── README.md            # Setup guide
```

11.1 Git Initialization Commands

```
mkdir smart-campus-helpdesk
cd smart-campus-helpdesk
git init
git add .
git commit -m "Initialize Unit 1 project structure"
git status
```

11.2 Why Standardized Repository Structure Matters

- Developers can quickly locate modules and tests.
- CI/CD workflows remain organized inside .github/workflows.
- Documentation and diagrams remain separate from application code.
- Container and deployment configuration can be version controlled.
- New team members can understand the project layout faster.

12. Verification, Testing & Acceptance Plan

Verification checks whether the implemented system satisfies the documented requirements. The acceptance plan connects test scenarios to expected outcomes before final release.

Test ID	Test Scenario	Expected Result	Status
T01	Login with correct and wrong credentials	Correct login succeeds; wrong password fails.	Planned
T02	Watson FAQ – Wi-Fi reset	Configured FAQ is answered correctly.	Planned
T03	Create ticket	Ticket is saved to Cloudant with OPEN status.	Planned
T04	Role access	Student cannot access admin portal.	Planned
T05	Screenshot upload	File is stored in Object Storage and linked by URL.	Planned
T06	Ticket routing	Ticket is assigned according to configured category/priority rule.	Planned
T07	Dashboard counts	Active/resolved/overdue information is displayed.	Planned

12.1 Acceptance Criteria

- Authentication works for defined roles.
- AI assistant answers configured FAQs.
- Tickets are stored with required fields.
- Unauthorized role access is blocked.
- Attachments are stored and linked.
- CI/CD can build and deploy the application.
- Basic user and staff workflows can be completed without missing required information.

12.2 Quality Checks

- Functional correctness
- Security and role access
- API/database performance
- Cloud deployment health
- User interface clarity
- CI/CD build reliability
- Monitoring and feedback readiness

13. Conclusion & Future Scope

The AI-Powered Smart Campus Helpdesk & Incident Management System provides a structured cloud-native approach for handling campus incidents. Unit 1 connects product engineering, user-centered thinking, requirements engineering, architecture, cloud services, Agile delivery, automated testing and continuous improvement.

13.1 Key Outcomes of Unit 1

- Clear campus business problem and justification.
- Defined user personas and stakeholder responsibilities.
- IBM Design Thinking stages and continuous Observe–Reflect–Make loop.
- Functional and non-functional requirements.
- Cloud architecture and Cloudant data design.
- Agile sprint plan and engineering roadmap.
- Repository structure and Git setup.
- Verification and acceptance plan.

13.2 Future Scope

- Email & SMS notifications
- Predictive hardware failure analytics
- Duplicate-ticket detection
- Knowledge-base recommendations
- Mobile-friendly interface
- Advanced SLA analytics
- More campus service categories and integrations

SUBMISSION SUMMARY

Project: AI-Powered Smart Campus Helpdesk & Incident Management System

Student: Nikhil Shidore

Department: Artificial Intelligence & Data Science (AIDS)

PRN: 2126UDSM2575

Subject: DevOps and Cloud Foundation (IBM)